



第十一章 软件架构质量

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第11章 软件架构质量

- 11.1 引言
- 11.2 软件架构与质量属性
- 11.3 软件架构质量指标
- 11.4 软件架构质量保障和评估方法
- 11.5 小结

11.1 引言

- 不同的设计师设计出来的架构存在物理差异，也存在质量差异。
- 有经验的架构设计师总能设计出高质量的软件架构。那么，软件架构的质量是什么？

11.1 引言

- 软件架构的质量也有内部质量和外部质量之分：
 - 一般认为开发态软件架构是软件的静态架构，难以在真实环境中进行实际的运行，其质量是内部质量，包含软件架构模型、数据、描述文档和视图的质量等；
 - 处于运行和维护演化过程的软件架构，称为运行态软件架构（动态架构），其质量是外部质量，包含基于该软件架构开发的系统的性能、可靠性、安全性等。

11.1 引言

- 讨论软件架构质量问题的意义：
 - 最终软件产品质量问题是当前软件开发发展过程的重要核心关注点之一
 - 问题发现的越早，解决问题的代价越小
 - 软件架构自身存在着很高的质量需求

11.1 引言

- 软件架构质量保障的好处：
 - (1) 对软件架构进行精确理解。
 - (2) 为相互冲突的目标划定优先级。
 - (3) 督促软件架构师更详细地编写软件架构文档。
 - (4) 发现项目之间交叉重用的可能性。
 - (5) 提高软件架构实践者的水平。
 - (6) 有益于该组织未来所从事的项目开发。

11.2 软件架构与质量属性

- 软件架构和软件产品质量之间的关系是相辅相成的
 - 一方面软件架构本身具有质量，高质量的软件架构设计会带来高质量的软件产品；
 - 另一方面，软件最终产品的质量可以间接地反映软件架构的质量，可以进一步指导软件架构的演化和优化等。

11.2 软件架构与质量属性

- 软件产品质量与架构之间关系
 - 性能：如何提高组件间的通信性能
 - 可用性：如何保证专用组件的可用性。专用组件,例如安全内核或认证服务器
 - 可靠性：如何通过使用冗余组件实现容错，提高可靠性
 - 安全性：如何保障组件的安全交互
 - 易用性：如何保障组件和架构的易用性问题
 - 可更改性：组件和架构的可更改性如何得到保障

11.2 软件架构与质量属性

- 软件产品质量与架构之间关系
 - 可移植性：组件是否可移植，架构是否可移植
 - 可重用性：组件间是否是松散耦合
 - 可集成性：组件/连接件接口是否统一，是否兼容
 - 可测试性：组件和连接件的测试难度如何，测试环境的配置是否比较困难等

11.2 软件架构与质量属性

表11.2 质量属性之间的正负关系

↕	性能↕	可靠性↕	可用性↕	安全性↕	易用性↕	可更改性↕	可移植性↕	可重用性↕	可集成性↕	可测试性↕
性能↕		-↕	↕	↕	-↕	-↕	-↕	↕	-↕	-↕
可靠性↕	↕		+↕	↕	+↕	+↕	↕	↕	↕	+↕
可用性↕	↕	+↕		↕	↕	↕	↕	↕	↕	↕
安全性↕	-↕	↕	↕		-↕	↕	↕	-↕	-↕	-↕
易用性↕	-↕	↕	↕	↕		↕	↕	↕	↕	-↕
可更改性↕	-↕	+↕	+↕	↕	↕		↕	↕	↕	+↕
可移植性↕	-↕	↕	↕	↕	-↕	+↕		+↕	+↕	+↕
可重用性↕	-↕	-↕	↕	-↕	↕	+↕	↕		+↕	+↕
可集成性↕	-↕	↕	↕	-↕	↕	+↕	+↕	↕		↕
可测试性↕	-↕	+↕	+↕	↕	+↕	+↕	↕	↕	↕	

11.3 软件架构质量指标

- 软件架构的质量指标：
 - **内部质量指标**是用来**直接**地评估软件架构自身的质量，包括软件架构文档的可读性、数据的一致性和兼容性、架构模型的完整性、软件架构的可重配置性、可维护性等；
 - **外部质量指标**是用来**间接**地评估软件架构的质量，这些指标其实都是基于该架构开发的最终软件系统的质量指标，这些指标不好的话，也可以间接反映软件架构存在缺陷

11.3.1 内部质量指标（1）

- 软件架构的内部质量是指描述软件架构文档、数据、图表和模型的质量，具体来讲，还指构成软件架构的组件、连接件、配置、数据和接口的质量。
- 软件架构典型的内部质量指标有（文档、数据、图表、模型）的可维护性、可重用性、可移植性、可集成性和可测试性等。

11.3.1 内部质量指标 (2)

- 可维护性 (Maintainability)
 - 软件可维护性主要指软件系统或组件在纠正错误，提升性能或其他属性，以及适应变化的环境的修改容易程度。
 - 改正性维护 (corrective maintenance) 是指改正在系统开发阶段已发生而系统测试阶段尚未发现的错误。
 - 适应性维护 (adaptive maintenance) 是指软件适应信息技术变化和管理需求变化而进行的修改。

11.3.1 内部质量指标（3）

- 可维护性

- 完善性维护（perfective maintenance）是为扩充功能和改善性能而进行的修改，主要是指对已有的软件系统增加一些在系统分析和设计阶段中没有规定的功能与性能特征。
- 预防性维护（preventive maintenance）为了改进应用软件的可靠性和可维护性，为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使应用系统适应各类变化而不被淘汰。

11.3.1 内部质量指标（4）

- 可重用性（Reusability）
 - 可重用性通常是指要合理地设计系统使得系统结构或其某些组件能够在未来的应用开发中可以重复使用的能力。它主要关注如下问题：
 - 在系统的不同地方使用不同的代码或组件来实现相同的功能
 - 使用多个类似的方法实现仅有细微差别的任务。
 - 使用几个系统来实现的特征或者功能，而不是共享或重用其他系统（多个系统或一个应用程序中的多个子系统）中的功能。

11.3.1 内部质量指标（5）

- 可移植性（Portability）
 - 可移植性是系统能够在不同计算环境（或平台）下运行的能力
 - 在软件架构中，对平台相关问题的封装表现为可移植性层面。可移植性层面是一组软件服务的集合，它使应用软件具有与其环境的抽象接口，将应用软件与环境的变化隔离开来。
 - 可移植性层面是应用信息隐藏设计原理的直接结果。

11.3.1 内部质量指标 (6)

- 可集成性 (Integrability)
 - 可集成性是使其他独立开发的系统组件能够与待开发系统协同运行的能力。
 - 可集成性取决于组件的外部复杂性、它们的交互机制和协议以及组件功能划分的清晰程度等，而这都是软件架构层次上的问题。
 - 可集成性还取决于组件接口的定义是否完整、合理等。

11.3.1 内部质量指标 (7)

- 可测试性 (Testability)
 - 可测试性是指通过测试（通常是基于运行的测试）使软件表露出缺陷的容易程度。
 - 可测试性与可观察性、可控制性有关。要能够对某个软件系统进行有效的测试，必须能够控制每个组件的内部状态及其输入，并能够观察其输出结果。
 - 系统的可测试性涉及若干个结构上的或软件架构上的问题：如在架构层次上编制文档的水平、对问题隔离的情况、系统使用信息隐藏原理的程度等。

11.3.2 外部质量指标（1）

- 软件架构的外部质量是指在软件系统运维过程中，软件系统体现出来的与软件架构有关的质量属性。
 - 性能
 - 可用性
 - 可靠性
 - 安全性
 - 易用性

11.3.2 外部质量指标 (2)

- 性能 (Performance)
 - 性能是指系统的响应能力,即要经过多少时间才能对某个刺激(事件)做出响应,或者在某个时间段内所能处理的事件的个数。
 - 性能主要关注如下问题:
 - (1) 客户端响应时间增加、吞吐量降低以及服务器资源过载使用。
 - (2) 内存消耗增加、以及过多的缓存未命中 (在缓存中不能找到需要数据) 。
 - (3) 数据库服务器处理增加。
 - (4) 网络带宽消耗增加。

11.3.2 外部质量指标 (3)

- 可用性

- 可用性 (Availability) 是指系统正常运行的时间比例。
- 可用性是通过两次故障之间的时间长度或系统崩溃的情况下系统能够恢复正常运行速度来衡量。

11.3.2 外部质量指标（4）

- 可用性主要关注如下的问题：
 - （1）物理层(如数据库服务器或应用服务器)发生故障或变得无响应，会导致整个系统运行失败
 - （2）如果系统不能及时处理由于网络配置或网络阻塞引起的大规模的负载，采用拒绝服务、阻止授权用户访问系统等中断系统操作。
 - （3）不恰当的资源使用会降低可用性。例如，过早的获取资源并长期持有会导致资源匮乏，并降低了处理其他并发用户请求的能力。

11.3.2 外部质量指标 (5)

- 可用性主要关注如下的问题：
 - (4) 应用程序中的错误或故障会导致系统范围内的失败。
 - (5) 频繁的升级会降低系统的可用性。
 - (6) 网络故障或导致应用程序的不可用。需要考虑如果处理不稳定的网络连接。
 - (7) 在应用程序中考虑信任边界并保证子系统采用了一定程度的访问控制或防火墙技术,同时也要考虑外延数据的有效性,来提高系统的弹性和可用性。

11.3.2 外部质量指标（6）

- 可靠性（Reliability）
 - 可靠性指的是系统能够保持正常运行的能力。可靠性可以分为两个方面：
 - （1）容错：其目的是在错误发生时确保系统正确的行为，并进行内部修复。
 - （2）健壮性：这里说的是保护应用程序不受错误使用和错误输入的影响，在遇到意外错误事件时确保应用系统处于已经定义好的状态。

11.3.2 外部质量指标（7）

- 可靠性主要关注如下的问题：
 - （1）系统崩溃或变得没有回应时，探测失效根源并自动启动失效备援，或者将负载转送到备份系统。
 - （2）输出不一致时，通过执行插桩探测性能缺陷根源，并通过系统输出相关信息。
 - （3）由于外部因素（例如网络不可用）导致系统失效时，寻找合适的方式处理不可靠的外部系统、失效的交互以及失效的事务等。

11.3.2 外部质量指标（8）

- 安全性（Security）
 - 安全性是衡量系统在向合法用户正常提供服务的情况下，阻止企图非授权使用，或者抗拒拒绝服务攻击（denial of service attack），并阻止信息泄露和丢失的能力。
 - 影响系统安全性的主要因素包括机密性，完整性和可用性。
 - 提高系统安全性主要通过授权、加密、审计及日志等手段来实现。

11.3.2 外部质量指标 (9)

- 安全性主要关注的问题有：
 - (1) 欺骗用户身份
 - (2) 由恶意输入引起的危害
 - (3) 数据篡改
 - (4) 拒绝用户行为
 - (5) 信息泄露及敏感数据丢失
 - (6) 由于DoS攻击导致的服务中断

11.3.2 外部质量指标（10）

- 易用性（Usability）
 - 易用性包含如下几个方面的意思：
 - （1）可学习性
 - （2）效率
 - （3）可记忆性
 - （4）错误避免
 - （5）错误处理
 - （6）满意度

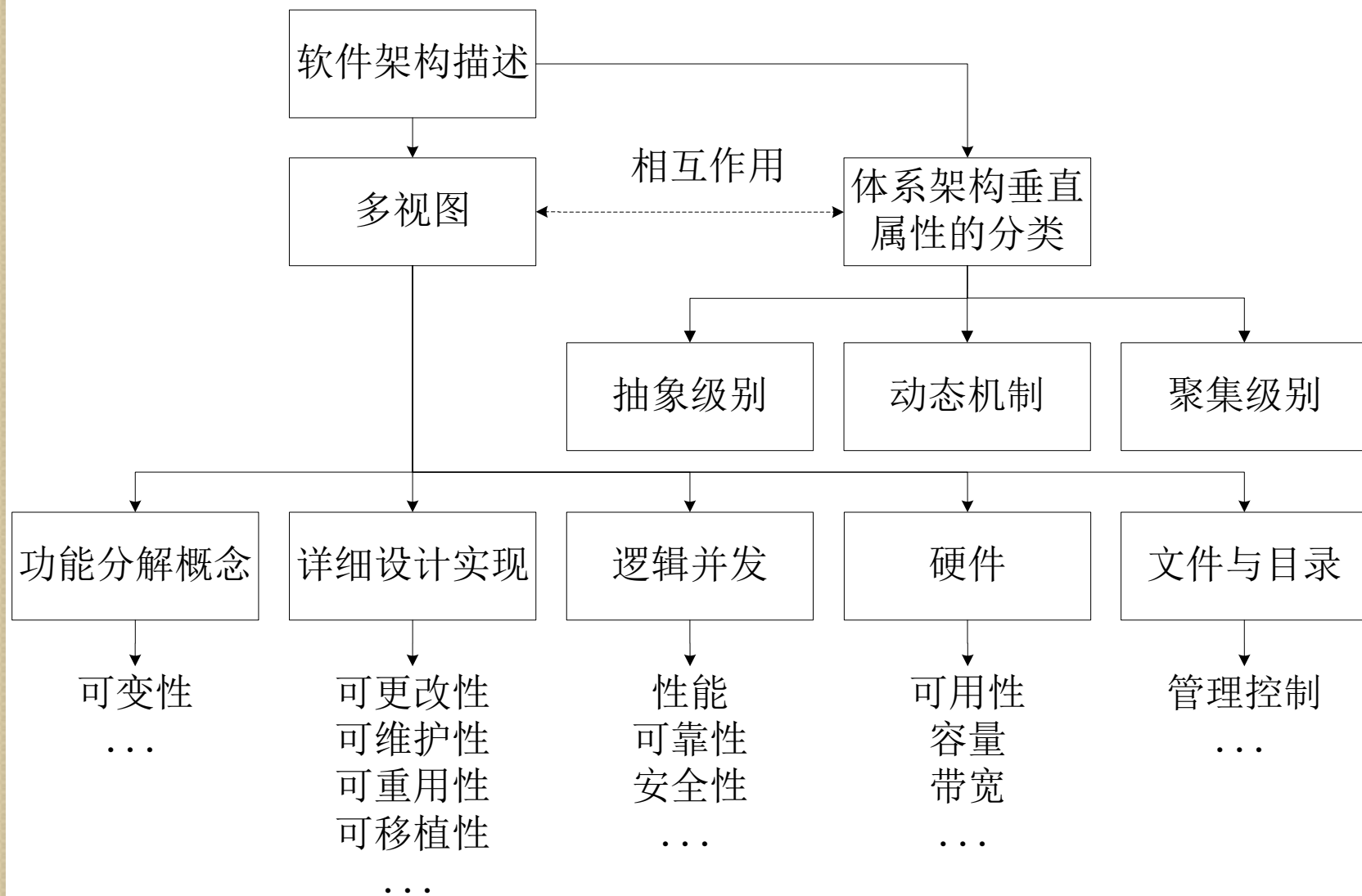
11.3.2 外部质量指标（11）

- 易用性主要考虑如下的问题：
 - （1）完成一项任务需要太多的交互（大量的点击），保证对屏幕输入流程及交互模式的设计最大限度地提高易用性。
 - （2）在多步骤的接口中存在不正确的流程步骤，考虑结合工作流来简化多步骤操作。
 - （3）数据元素和控制没有很好进行分类，选择合适的控制类型并采用公认的UI设计模式对控制符及内容进行设计。
 - （4）给用户的反馈非常薄弱，特别是在应用程序发生错误，异常或没有响应的情况下，考虑采用相关技术最大限度地提高用户交互性。

11.4 软件架构质量保障和评估方法

- 软件架构评估的目的是判断该架构是否实现了风险承担者的质量需求。
 - 首先，把视图进行细化，建立视图细节与特定质量属性之间的关系；
 - 然后，需要进行评估前的准备、识别风险承担者、组织评估小组、确定评估时间和选择合适的评估技术等工作，以便高效的完成软件架构评估工作。

11.4 软件架构质量保障和评估方法



11.4.1 评估准备

- 软件架构评估作为软件开发过程的一个重要步骤，首先需要建立规范的评估文档（类似于软件测试中测试用例），这类文档主要供评估人员和风险承担者参阅和交流

11.4.1 评估准备

表 11.3 评估文档的组成

主题	描述
方法目标	该方法的特定目标是什么
质量属性	对哪些质量属性进行评估
软件架构描述	评估关系到哪些软件架构的视图
评估技术	评估方法中包含哪些技术
参与者	评估过程中涉及系统的哪些参与者
评估中的活动	以何种顺序, 何种方式, 何种评估技术完成了该方法的特定目标, 该方法描述了什么结果
方法验证	是否在实际中得到了验证

11.4.2 风险承担者

- 软件系统架构涉及到很多人的利益，这些人都对软件架构施加各种影响，以保证自己的目标能够实现。
- 风险承担者（Stakeholder）就是在该架构及根据该架构开发的系统中有既得利益的人
- 在任何一次的软件架构评估中，准确地获得风险承担者对软件架构的预期目标是至关重要的一步

11.4.2 风险承担者

- 系统生产者：架构师、开发人员、维护人员、测试人员、集成人员、标准专家等等；
- 系统消费者：客户、最终用户、应用开发者、任务专家
- 系统服务人员：系统管理员、网络管理员，服务代表
- 接触系统与系统交互的人：团体代表、系统架构师、设备专家

11.4.3参与者

- 评估参与者会构成一个评估团队 (Evaluation Team) ，人员包括评估小组负责人、评估负责人、场景记录员、进展记录员、过程观察员、过程监督员和提问者等。

11.4.4 评估时机

- 架构评估的时机一般在架构明确之后，具体实现开始之前。
- **早期评估**主要发生在初期阶段，即完成高层次的架构以及部分高优先级的架构决策。
- **中期评估**主要发生在架构设计实施部分精化之后。
- **后期评估**主要发生在系统已经被完整地设计、实现并部署之后。

11.4.5 评估技术

- 基于问卷调查或检查表的软件架构评估技术：
 - 问卷调查是一系列应用到各种软件架构评估的相关问题，其中有些问题可能涉及到软件架构的设计决策；有些问题涉及到软件架构的文档。
 - 检查表中也包含一系列比调查问卷更细节和具体的问题，它们更趋向于考察某些关心的质量属性。

检查表示例

Y: 是 TBD: 不确定 N: 不是 NA: 不适用		备注
检查项	Y/TBD/N/NA	
清晰性		
系统的目标是否已定义?		
是否对关键术语和缩略语进行定义和描述?		
所使用的术语是否和用户/客户使用的一致?		
需求的描述是否清晰, 不含糊?		
是否有对整套系统进行功能概述?		
是否已详细说明了软件环境 (共存的软件) 和硬件环境 (特定的配置)?		
如果有会影响实施的假设情况, 是否已经声明?		
是否已经对每个业务逻辑进行输入、输出以及过程的详细说明?		
完整性		
是否列出了系统所必须的依赖、假设以及约束?		
是否对每个提交物或阶段实施都进行了需求说明?		

11.4.5 评估技术

- 基于问卷调查或检查表的软件架构评估技术:
- 优点:
 - 这一评估方式比较自由灵活，可评估多种质量属性，也可以在软件体系结构设计的多个阶段进行。
- 缺点:
 - 由于评估的结果很大程度上来自评估人员的主观推断，因此不同的评估人员可能会产生不同甚至截然相反的结果，而且评估人员对领域的熟悉程度、是否具有丰富的相关经验也成为评估结果是否正确的重要因素。

11.4.5 评估技术

- 基于问卷调查或检查表的软件架构评估技术：
 - 尽管基于调查问卷与检查表的评估方式相对比较主观，但由于系统相关的人员的经验和知识是评估软件体系结构的重要信息来源，因而它仍然是进行软件体系结构评估的重要途径之一。

11.4.5 评估技术

- 基于场景的软件架构评估技术:
 - 在软件架构评估中一般采用激励 (Stimulus)、环境 (Environment) 和响应 (Response) 三方面对场景进行描述。

11.4.5 评估技术

- 基于度量的软件架构评估技术：
 - (1) 要建立质量属性和度量之间的映射原则，即确定如何从度量结果推出系统具有何种质量属性；
 - (2) 从软件架构文档中获取度量信息；
 - (3) 根据映射原则分析推导出系统的某些质量属性。

11.4.5 评估技术

- 基于度量的软件架构评估技术：
 - (1) 要建立质量属性和度量之间的映射原则，即确定如何从度量结果推出系统具有何种质量属性；
 - (2) 从软件架构文档中获取度量信息；
 - (3) 根据映射原则分析推导出系统的某些质量属性。

例如：实验室项目C语言架构评估

指标名称	指标解释	度量方法	度量值
易理解性	易理解性度量的是开发人员对于设计和现有工程文件理解的难易程度，值越高，设计和工程文件越易于理解，越利于开发人员开展演进活动。	易理解性与模块中注释的标注有关系。模块中注释标注的越完善，占模块总数的比例越高，开发人员对模块的理解越容易，设计的易理解性越高。调用关系越复杂，设计修改难度越大。	0.8
可修改性	该指标度量的是对设计进行修改的难度，在本模型中，特指对设计改正和改进活动实施的难度，值越高，实施难度越低。	可修改性要与设计中各模块间的依赖关系复杂程度有关系，依赖关系越复杂，设计修改难度越大	0.751
可替换性	该指标度量设计中的模块被具有相似功能的模块替换的难度，以及删除特定功能模块的难度，值越高，替换/删除难度越小。	一个函数被其他函数调用的越多，替换或删除该函数造成的影响越大，替换或删除的难度也越大。函数被其他函数调用的关系反映在函数的入度，函数的入度越高，可替换性越低。	0.3481
可拓展性	该指标度量的是对设计进行扩展的难度，值越高，扩展难度越低。	在对设计进行扩展时，通常是基于现有API方法进行操作，所以该指标的度量与API方法占方法总数的比例有关系，该比例越高，则设计的可扩展性越高。	0.1
易测试性	该指标度量对该软件设计进行测试的难度，值越高，测试难度越小。	在对软件进行测试时，如果一个函数依赖于其他函数，通常需要将其一同测试，这样增加了测试的难度，因此该指标的度量与每个函数的出度直接。相关	0.92

11.4.5 评估技术

	基于问卷调查 或检查表的评 估技术	基于场景的 评估技术	基于度量的 评估技术
通用性	通用或特定领 域	特定领域	通用或特 定领域
评估者对待 评估架构的 掌握程度要 求	简单了解	比较熟悉	精确掌握
客观性	主观	比较主观	比较客观
评估时间	早期,中期	中期	后期

11.4.6 软件架构评估的收益与成本

- 软件架构评估常见的一些收益：
 - (1) 把各个风险承担者召集到一起。
 - (2) 迫使对具体的质量目标做出清楚的表述。
 - (3) 为相互冲突的目标划分优先级。
 - (4) 迫使对软件架构做出更清楚的解释。
 - (5) 提高架构文档的质量。
 - (6) 发现项目之间交叉重用的可能性。
 - (7) 提高架构实践水平。

11.5 小结

- 本章着重描述了软件架构质量，包括外部质量和内部质量两类，因为从严格意义上讲，软件架构质量保障是属于软件开发早期阶段的质量保障，它不仅包含软件架构自身的质量（是指软件架构模型、文档、数据和图表的质量），也包含基于该软件架构的最终软件产品的质量。